

Context

PDE-constrained optimisation is the problem of choosing an input to a physical system, described by a partial differential equation, so that its solution matches some target as closely as possible. This can be used in a wide range of applications, such as configuring heat sources or initialising weather forecasts. In each case the unknown is a *function* on a domain, not a vector of numbers, and this is what we are solving for.

Firedrake [1] is an open-source Python library for solving PDEs by the finite element method, and pyadjoint [2] is its adjoint extension. Together, these let a user write such problems in code that closely mirrors the mathematical formulation; the optimisers come from PETSc TAO. There is a catch: the natural way to measure the size of a function is a Hilbert-space inner product, not the Euclidean one. Using the wrong choice makes iteration counts grow polynomially as the mesh becomes finer and more uneven.

The finite element method

Computers only work with finite lists of numbers, not with continuous functions. The **finite element method** (FEM) overcomes this issue by dividing the domain into small polygons (a *mesh*) and approximating the unknown function as a low-degree polynomial on each cell. FEM works on domains of arbitrary shape, and lets the user place small cells wherever accuracy is needed and larger cells elsewhere. So, real-world meshes are almost never uniform.

The mesh-dependence phenomenon

Every gradient-based optimiser uses *inner products* to measure the size of gradients. Schwedes et al. [3] show that on non-uniform meshes, using the Euclidean inner product makes iteration counts grow polynomially with $h_{\max}/h_{\min}$ (the largest cell size divided by the smallest). However, using the natural L^2 inner product on the function space, which accounts for cell sizes, keeps our iteration counts bounded.

The fix is the *Riesz map* $\mathcal{R}$, a linear operator that converts a raw gradient into a properly scaled update direction. In the discrete L^2 setting,

$$\mathcal{R}_{L^2} = M_h^{-1},$$

where M_h is the *mass matrix*, with entries $(M_h)_{ij} = \int \phi_i \phi_j, \mathrm{d}x$ encoding L^2 inner products between finite-element basis functions ϕ_i .

Test problem

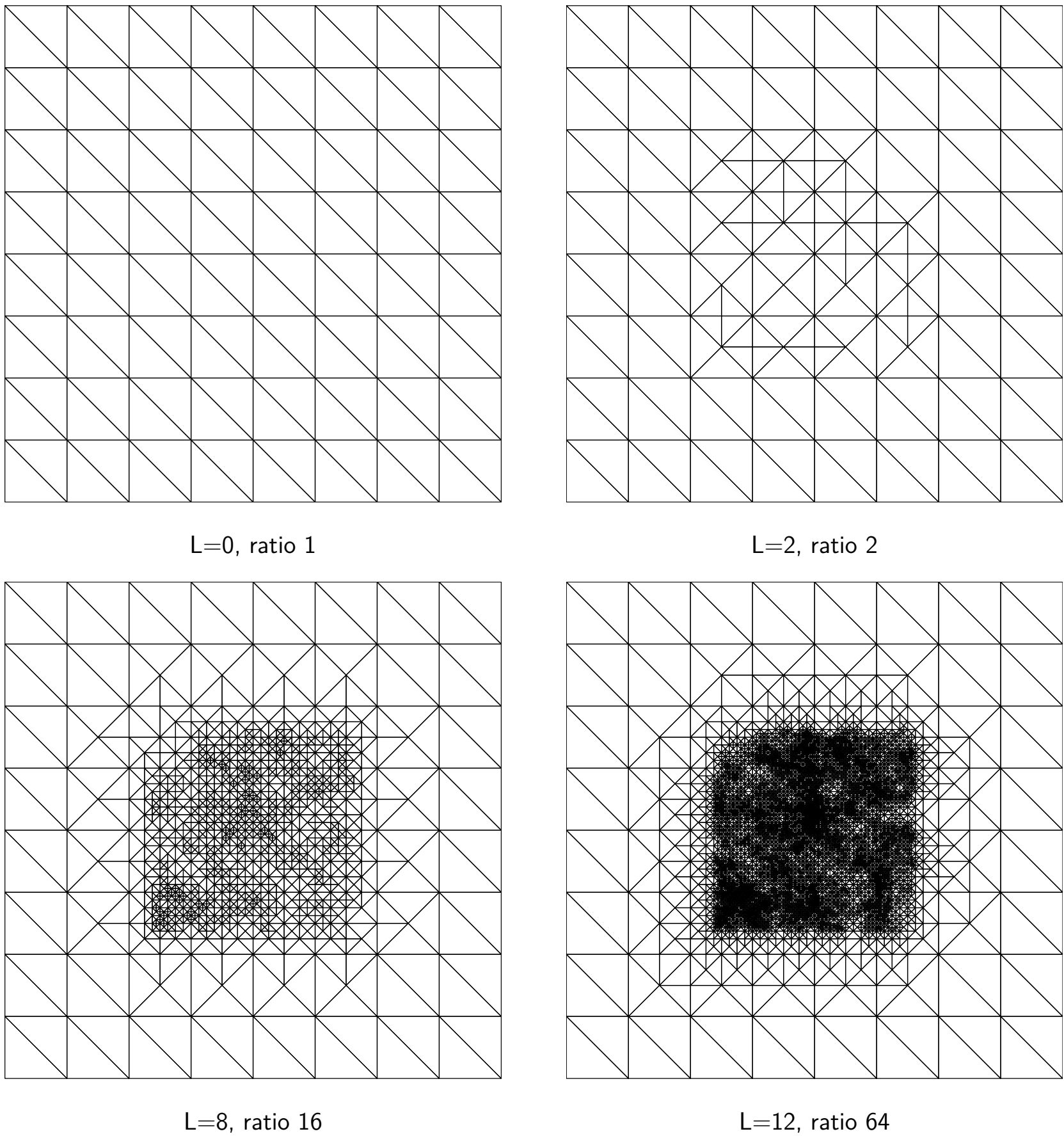
We use the classic *Poisson optimal control problem* as our test problem - it is simple enough so that the maths is clear, but still rich enough to show mesh dependence clearly. It reads: on the unit square $\Omega = (0, 1)^2$, we want to find a source m so the state u matches a target d :

$$\tilde{J}(m) = \frac{1}{2} \|u - d\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m\|_{L^2(\Omega)}^2, \quad -\Delta u = m, \quad u = 0 \text{ on the boundary } \partial\Omega,$$

with $d = \sin(\pi x) \sin(\pi y)$ and $\alpha = 10^{-4}$. The gradient of $\tilde{J}$ is computed automatically by pyadjoint and passed to **PETSc TAO** (Toolkit for Advanced Optimisation), a library of optimisation solvers that uses the gradient to iterate towards the optimal source m .

Meshes

We need a way to generate meshes, so we adapt the method of [3, Sect. 2.3.2.1] to produce a set of increasingly non-uniform meshes. The largest cells stay the same size, whereas the smallest cells shrink geometrically, so $h_{\max}/h_{\min}$ doubles every two levels.



Objectives

- 1. **Check whether Schwedes’ mesh dependence problem is still present** in the modern Firedrake, pyadjoint, and PETSc TAO stack, ten years on. Reproduce their experiment and determine which TAO solvers work correctly and which do not.
- 2. **Fix the mesh-dependence deeper in the solver.** Some optimisers solve a linear system inside every step, and that inner solve inherits the same mesh-dependence. Applying the Riesz map there too removes it - a contribution not covered by Schwedes.
- 3. **Deliver the fix as a pyadjoint feature.** Users should get correct mesh-independent behaviour from a single options entry, without needing to know about Riesz maps.

Reproducing the experiment (outer L-BFGS)

L-BFGS is a widely-used quasi-Newton optimiser: it builds up a low-rank approximation of the Hessian across iterations and updates the solution using this approximation. Two L-BFGS optimisers run on the same problem, on the same mesh sequence, to the same tolerance. The only difference is the inner product each uses internally.

Inner product used	L=0	L=2	L=4	L=6	L=8	L=10	L=12	L=14
Realised $h_{\max}/h_{\min}$	1	2	4	8	16	32	64	128
Euclidean ℓ^2	13	39	71	104	205	401	701	1269
Function-space L^2	6	8	6	8	7	8	8	6

The ℓ^2 row grows nearly $100\times$ across the sweep, matching the polynomial dependence predicted by Schwedes. The L^2 row stays bounded between 6 and 8. TAO’s `lmvm` solver reproduces the flat row once its default limited-memory history is raised from five to one hundred.

Preconditioning

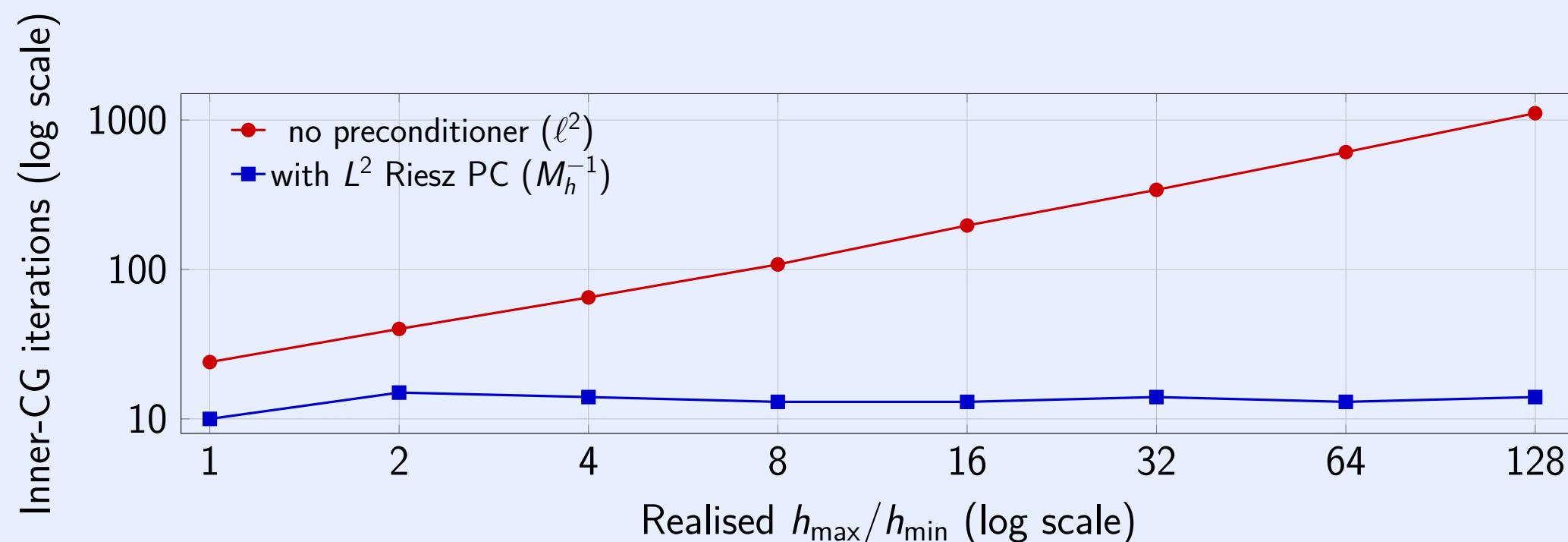
Some iterative solvers, like *conjugate gradient* (CG), can converge very slowly when the underlying matrix is badly conditioned. A *preconditioner* P transforms a difficult linear system $Ax = b$ into an easier one,

$$P^{-1}Ax = P^{-1}b,$$

whose matrix $P^{-1}A$ is close to the identity. The iterative method then converges much faster - often in a number of iterations independent of problem size. For the reduced Hessian on non-uniform meshes, the Riesz map M_h^{-1} is exactly the right preconditioner: it cancels the cell-size scaling that would otherwise ruin the conditioning.

New result: preconditioning the inner CG of TAO/NLS

TAO has many optimisers. L-BFGS (previous section) builds a Hessian approximation across iterations, so a correct choice of inner product is enough to make it mesh-independent, and it does not need a preconditioner. **NLS** (Newton Line Search) is Newton’s method: at each outer step it solves the exact reduced-Hessian system for a search direction, using inner CG. On our test problem NLS takes only two outer iterations, but each inner CG grows in iteration count with mesh non-uniformity unless preconditioned. The discrete Riesz map M_h^{-1} removes this growth completely.



The uncorrected inner solve inherits the same weakness as the outer optimiser: iteration counts grow nearly $50\times$ across the mesh sweep. The L^2 Riesz-map preconditioner does at the Krylov level what changing the inner product does at the optimiser level, restoring mesh-independence completely. This connection is pointed at by [3, Sect. 1.3.3] using [4], but it is not tested experimentally in the book.

Work plan

- **Ship the fix as a pyadjoint feature.** Users should be able to enable the Riesz preconditioner with a single options entry; the inner product should be read off the Control. This is the main remaining goal.
- **Extend to H^1 and other Hilbert spaces on the control**, and characterise when each Riesz map is the correct preconditioner.
- **Extend mesh-independence across TAO’s optimisers.** Newton and L-BFGS now behave correctly; others, such as `bqnls`, do not yet. Bringing the whole solver family in line lets TAO be used with confidence on function-space problems.

References

[1] Rathgeber et al. (2016). Firedrake: automating the finite element method by composing abstractions. *ACM TOMS* 43(3).
[2] Mitusch, Funke, Dokken (2019). dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *JOSS* 4(38).
[3] Schwedes, Ham, Funke, Piggott (2017). *Mesh Dependence in PDE-Constrained Optimisation*. SpringerBriefs.
[4] Kirby (2010). From functional analysis to iterative methods. *SIAM Review* 52(2).